Adaboost Analysis:

[6]:
```
from sklearn.model_selection import train_test_split
X_train,X_test,y_train,y_test=train_test_split(independent,dependent,test_size=0.30,random_state=0)
```

[24]:
```
from sklearn.ensemble import AdaBoostRegressor
regressor = AdaBoostRegressor(random_state=0, n_estimators=7000)
```

[25]:
```
regressor.fit(X_train,y_train)
```

```
C:\Anaconda3\Lib\site-packages\sklearn\utils\validation.py:1406: DataConversionWarning: A column-vector
se change the shape of y to (n_samples, ), for example using ravel().
  y = column_or_1d(y, warn=True)
```

[25]:
```
▾ AdaBoostRegressor
▶ Parameters
```

[26]:
```
y_pred = regressor.predict(X_test)
```

[27]:
```
from sklearn.metrics import r2_score
r_score=r2_score(y_test,y_pred)
r_score
```

[27]:
```
0.9372968179031409
```

Adjusting the n_estimators till 7000 shows the value of r_score as 0.937

Trying with XG_Boost:

[22]:
```
regressor=xgb.XGBRegressor(
    eval_metric='rmsle',
    learning_rate=0.1,
    n_estimators=500,
    max_depth=6,
    subsample=0.8,
    colsample_bytree=0.8,
    gamma=0.2,
    reg_alpha=0.1,
    reg_lambda=1,
    random_state=42)
```

[23]:
```
regressor.fit(X_train,y_train)
```

[23]:
```
▾ XGBRegressor
▶ Parameters
```

[24]:
```
y_pred = regressor.predict(X_test)
```

[25]:
```
from sklearn.metrics import r2_score
r_score=r2_score(y_test,y_pred)
r_score
```

[25]:
```
0.917903482913971
```

Getting R score value with adjustments made to learning_rate alone could get maximum of 0.917

LG_Boost:

Loss	**criterion**	**n_estimators**	**R_score**
squared_error	*friedman_mse*	100	0.9226
squared_error	squared_error	100	0.9226
absolute_error	*friedman_mse*	100	0.8816
absolute_error	squared_error	100	0.8816
huber	*friedman_mse*	100	0.9220
huber	squared_error	100	0.9220
quantile	*friedman_mse*	100	0.8238
quantile	squared_error	100	0.8238

Getting r_score value as 0.9226

Conclusion: Comparing all the 3 could see the r_score value of Adaboost is good aquiring r_score as 0.937, So deploying the same.